

APUNTES DE RUST

DECLARACIÓN DE VARIABLES

La declaración de la variable es inversa a c:

Nombre_de_variable: tipo

var1: f32

var2: f64

Los tipos en comparativa:

int	-> i32	char	-> char (son 4bytes unicode no solo ascii)
-----	--------	------	--

long	-> i64	bool	-> bool
------	--------	------	---------

float	-> f32	char*	-> &str
-------	--------	-------	---------

double	-> f64	std::string	-> String
--------	--------	-------------	-----------

U-Int -> u32

U-long -> u64

Para asignar un valor a una variable seria:

```
let a = 32; //no habría una declaracion previa tipo a: i32, y Rust deduce lo que es.\
```

```
let a = 32 as f32 //le indicamos que es un float
```

o

```
let a: i32 = 42;
```

LET es para hacer que no pueda variar (NO PUEDE SER GLOBAL). **LET**
MUT para que varie (Hay que poner **let** delante), y **CONST** para constante (si puede ser global)

```
let mut i = 0; //tipico para hacer while ya que la i variará. No global. En runtime
```

```
const i: i32 = 0; //no puede mutar NUNCA en compilacion y puede ser global. Necesita ser declarado el tipo. y se crea en tiempo de compilación
```

Si a una variable le ponemos **'_'** delante, esta variable aunque no se use, el compilador no se quejará de ella:

```
let _num: f32 = 0.4; //aunque no la use el compilador no lanzara un warning
```

MATRICES

Podemos usar matrices como en C:

```
let x: [i32; 3] = [1, 2, 3];
```

```
//let x = [1, 2, 3]; puede ser así también y el deduciría el tipo.
```

```
println!("{}", x[1]); //imprimiría 2, pero es arriesgado
```

```
println!("{}", x[7]); //Entraría en modo "panic" que saldría de forma segura. No soltaría basura
```

Para hacerlo de forma segura se puede usar

```
x.get(1); //devolvería "Some(2)" en mayúsculas la S
```

```
x.get(7); //devolvería "None" en mayúsculas la N
```

Por lo tanto para usarlo por que un `x.get(1)` devolvería en pantalla un `some(2)` con un `println!` tenemos que hacer:

```
if let Some(i) = x.get(1) {  
    println!("{}", i);  
} else {  
    println!("no existe");  
}
```

O en formato Rust:

```
match x.get(2) {  
    Some(i) => println!("{}", i),  
    None    => println!("No existe"),  
}
```

VARIABLES GLOBALES

El tener una variable global en Rust es un problema, ya que puede haber dataraces para modificarla así que tiene que estar protegida.. si no no deja.

La forma clásica es:

```
static mut A: i32 = 0; //variable global
```

```
fn main(){  
    if true{
```

```

    unsafe{ A = 10;} //no se recomienda pero permitido. unsafe le dice el compilador que confie en el
programador.
}
}

```

Unsafe es un peligro.. así que usamos **atomic** para no provocar data races:

```

use std::sync::atomic::{AtomicI32, Ordering}

static A: AtomicI32 = AtomicI32::new(0); //no hace reserva de memoria. Solo inicializa a cero

fn main() {
    if true {
        A.store(10, Ordering::Relaxed); //Relaxed es el nivel más débil del ordenamiento atómico.
        solo
        me importa modificar el valor, no el orden del tiempo en que se haga.
    }
}

```

ESTRUCTURAS

Rust no tiene objetos ni clases, pero sí que puede acercarse a ellos con las estructuras. En Rust todo es privado que se declare. Para hacerlo accesible hay que ponerle la palabra **pub** delante. No solo a la estructura.

(fichero.rs)

```

pub struct Prueba{ //debe ser CamelCase P mayúscula
    pub var1: i32,
    pub var2: char, //si no son pub estas variables no se puede acceder a ellas
}

```

Importando el **fichero.rs** como

(fichero: main.rs)

```

mod fichero; //importa el fichero fichero.rs

fn main(){
    let a = fichero::prueba { var1: 42, var2: 'a'};
}

```

Ahora imaginemos que tenemos un archivo.rs que tiene esto:

(fichero.rs)

```
pub struct Prueba{ //para implementar el getter tiene que ser la estructura pub
    x: i32,
}

pub fn test() -> Prueba{ //-> devuelve un tipo Prueba (estructura)
    Prueba {x: 56}
}
```

No habría problema en darle valor dentro de fichero.rs PERO si queremos acceder a el, tendríamos que hacer un metodo de la estructura en el propio archivo fichero.rs:

```
impl Prueba{
    pub fn get_x(&self) -> i32{ //tiene que ser pub tambien. -> i32 = devuelve un int.
        //&self es como los objetos.. o this en c++ va implicito
        (self) lo movería el valor
        (&self) es inmutable
        (&mut self) lo podría cambiar.
        self.x //IMPORTANTE!!! SI NO TIENE ';' ES UN RETURN IMPLICITO!!!
    }
}
```

En el main.rs:

```
mod fichero;

fn main (){
    let a = fichero::test(); //llama a la funcion externa en fichero y le da el valor 56
    println!("{}", a.get_x());
}
```

FUNCIONES

Las funciones en Rust estan divididas entre las que no devuelven nada y las que sí.

NO DEVUELVE:

```
fn funcion(num1: &i32){ //referenciada
    println!("El numero es: {}", num1);
}
```

DEVUELVE:

```
fn suma(num1: &i32, num2: &i32) -> i32{ //-> lo que devuelve
    *num1 + *num2 //es importante que no tenga ; ya que este es un return oculto, sin ese punto y coma.
    //Hay que derreferenciar, ya que son referencias (direcciones de memoria)
}
```

Para llamarlas:

```
funcion(&n);
suma(&x, &y);
```

Para cambiar el valor de un numero:

```
fn cambia(n: &mut i32){
    *n = 42;
}
```

y para llamarla:

```
cambia(&mut x);
```

FUNCIONES CLOSURE |parametros|

Hay veces que queremos pasar una funcion mas limpiamente o mas corta. Se puede definir la normal como:

```
fn sumar (x: i32, y: i32) -> i32{
    x + y
}
```

Pero podemos hacerla Closure:

```
let sumar = |x, y| x + y;
```

y se la llama como:

```
let resultado = sumar(2, 3);
```

esto mismo se puede hacer en una unica linea asi:

```
let sumar = (|x, y| x + y)(2, 3);
```

SOBRECARGA DE OPERADORES

Rust tiene sobrecarga de operadores, y la forma de declararlos es en el `impl` de tal forma:

```
use std::ops::Add; //importa el trait (el operador) de Add. Si no se puede usar
```

```

struct Vector3{...}

impl Add for Vector3{
    type Output = Vector3; //el resultado de la operacion v1 + v2 sera un Vect3
    fn add(self, other: Vector3) -> Vector3{...}
}

```

La sobrecarga es con nombre mayúscula, y la función en minúscula pero llamada igual. La lista es la siguiente para las sobrecargas (entre paréntesis el nombre a la función):

<code>+</code> -> <code>Add</code> (<code>add</code>)	<code>+=</code> -> <code>AddAssign</code> (<code>add_assign</code>)
<code>-</code> -> <code>Sub</code> (<code>sub</code>)	<code>-=</code> -> <code>SubAssign</code> (<code>sub_assign</code>)
<code>*</code> -> <code>Mul</code> (<code>mul</code>)	<code>*=</code> -> <code>MulAssign</code> (<code>mul_assign</code>)
<code>/</code> -> <code>Div</code> (<code>div</code>)	<code>/=</code> -> <code>DivAssign</code> (<code>div_assign</code>)
<code>%</code> -> <code>Rem</code> (<code>rem</code>)	<code>%=</code> -> <code>RemAssign</code> (<code>rem_assign</code>)
<code>==</code> -> <code>PartialEq</code> (<code>eq</code>)	<code>!=</code> -> <code>PartialEq</code> (<code>ne</code>)
<code>[]</code> -> <code>Index</code> (<code>index</code>)	<code>[] =</code> -> <code>IndexMut</code> (<code>index_mut</code>)
<code>-x</code> -> <code>Neg</code> (<code>neg</code>)	<code>!x</code> -> <code>Not</code> (<code>not</code>)
<code>&</code> -> <code>BitAnd</code> (<code>bitand</code>)	<code>&=</code> -> <code>BitAndAssign</code> (<code>bit_and_assign</code>)
<code> </code> -> <code>BitOr</code> (<code>bitor</code>)	<code> =</code> -> <code>BitOrAssign</code> (<code>bit_or_assign</code>)
<code>^</code> -> <code>BitXor</code> (<code>bitxor</code>)	<code>^=</code> -> <code>BitXorAssign</code> (<code>bit_xor_assign</code>)
<code><<</code> -> <code>Shl</code> (<code>shl</code>)	<code><<=</code> -> <code>ShlAssign</code> (<code>shl_assign</code>)
<code>>></code> -> <code>Shr</code> (<code>shr</code>)	<code>>>=</code> -> <code>ShrAssign</code> (<code>shr_assign</code>)
<code>*x</code> -> <code>Deref</code> (<code>deref</code>)	<code>*x=</code> -> <code>DerefMut</code> (<code>deref_mut</code>)
<code>x..y</code> -> <code>Range</code> (no hay)	<code>()</code> -> <code>Fn</code> , <code>FnMut</code> , <code>FnOnce</code> (<code>call</code>)
<code><, <=, >, >=</code> -> <code>PartialOrd</code> (<code>partial_cmp</code>)	

Para alterar el orden por ejemplo en una multiplicacion de un Vector3 por ejemplo seria:

```

2 * v1

impl Mul<Vect3> for i32{
    type Output = Vect3
    fn mul(self, other: Vect3) -> Vect3{
        return Vect3 {x: self * other.x, y: self * other.y, z: self * other.z}; //hay return, entonces hay ;
    }
}

```

```

}
v1 * 2
impl Mul<i32> for Vect3{
    type Output = Vect3;
    fn mul(self, n: i32) -> Vect3{
        Vect3 {x: self.x * n , y: self.y * n, z: self.z * n} //no hay return no hay ;
    }
}

```

El formato es:

```
impl Trait<RightHandElement> for LeftHandElement
```

entonces el self es del tipo del LeftHandElement

WHILE

El bucle while es:

```

let mut i: u32 = 1;
while i < 10 && i > 0{
    ....
    i += 1;
}

```

La condición nunca va entre paréntesis para evitar la confusión de C ya que en rust la condición es una expresión booleana. Esto es un error:

while 5 o while (5)

LOOP

loop es infinito a no ser que se saque

```

loop {
    codigo();
    if !condicion {
        break; // asi sale
    }
}

```

```
}
```

FOR

El for en Rust es bajo iteradores

```
for elemento in iterable {...}
```

Por ejemplo:

```
for i in 0..10{ ... } //i va de 0 a 9
```

o con matrices:

```
let v = [1, 2, 3];
```

```
for i in v{ ... }
```

COLLECTIONS (contenedores)

Como en C++ tenemos contenedores:

1. VECTOR DINAMICO:

```
let mut v = Vec::new();
```

```
v.push(1); //mete el 1
```

```
v.push(2); //mete el 2 detrás del 1.
```

```
v.pop(); // quita el 2 (el ultimo) Devuelve un Option<T>
```

Para acceder a los datos se puede usar:

```
v[i]; //puede ser panic
```

```
v.get(i); //seguro.
```

El tamaño es con:

```
v.len();
```

Se puede usar iteradores:

```
let mut it = v.iter();
```

```
it.next(); //es el siguiente.
```

```
it.next_back(); es el anterior.
```

Los dos devuelven Option<&T> por lo tanto para usarlos:

```
if let Some(x) = it.next(){...}
```


2. ARRAYS

3. STRING

4. HASHMAP

5. VECDEQUE

6. HASHSET

7. BTREEMAP